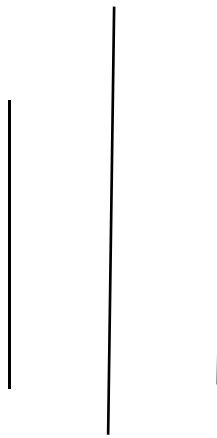




Tribhuvan University
Faculty of Humanities and Social Science
SWASTIK COLLEGE



Course: Mobile Programming(CACS351)

Title : Smart Contact Management Android Application (V1)

Submitted To:

Sujan Poudel
Lecturer, BCA Department
Swastik College, Bhaktapur

Submitted By:

Safalta Shrestha
(6-2-1131-47-2022)

5 April 2026

Table of Contents

Smart Contact Management Android Application (V1)	1
1. Introduction	1
2. Objectives The main objectives of Version 1 are:	1
3. Scope of Version 1	1
4. Tools and Technologies Used	2
5. Application Architecture (V1)	3
6. Project Folder Structure	3
7. Git Workflow and Version Control Strategy	4
8. Implementation Details	5
9. Screenshots	6
10. Conclusion	6
11. Future Work	7

Smart Contact Management Android Application (V1)

1. Introduction

Mobile applications are essential for simplifying everyday tasks and improving productivity. A Contact Management Application allows users to efficiently store, manage, and access contact details on a mobile device.

This project, Smart Contact Management Android Application (V1), developed for the Mobile Programming (CACS351) course, has been built to a production-ready standard. It integrates fundamental Android development concepts with a clean and intuitive interface based on Google's Material Design principles.

This version emphasizes efficient local data handling, complete CRUD operations, smooth and responsive performance, and well-organized project structure maintained through Git version control.

2. Objectives

The main objectives of Version 1 are:

- To develop a fully functional Android application using Android Studio.
- To implement complete CRUD (Create, Read, Update, Delete) operations for contact management.
- To store contact data locally using SQLite database.
- To display contact information efficiently using RecyclerView and MaterialCardView.
- To provide an intuitive and validation-enabled user interface using Material Design components.
- To perform database operations asynchronously to ensure smooth user experience.
- To follow proper Git version control practices during development.

3. Scope of Version 1

Included in Version 1

- **Offline Contact Storage:** Contacts are stored locally on the device using SQLite.
- **Full CRUD Functionality:** Users can add, view, edit, and delete contacts.
- **Expanded Data Model:** Each contact includes:
 - First Name
 - Last Name
 - Phone Number
- **Modern UI/UX Design:** Implementation of Activities, RecyclerView, FloatingActionButton, Standard Android Views for a polished interface.
- **Contact Detail Screen:** Read-only screen for viewing full contact details before editing.
- **Search Functionality:** Real-time filtering by name.
- **Asynchronous Database Operations:** Database operations are executed in background threads to avoid UI blocking.
- **Database Schema Migration:** Updated database version to support the expanded data model.

Excluded from Version 1

- Integration with system phone contacts.
- Caller identification functionality.
- Cloud synchronization and online backup.

4. Tools and Technologies Used

Category	Technology
Programming Language	Java
IDE	Android Studio
Platform	Android
Database	SQLite
UI Components	Activities, RecyclerView, Material Design Components
Concurrency	java.util.concurrent.Executors
Version Control	Git
Minimum SDK	API 21

5. Application Architecture (V1)

Version 1 follows an Activity-centric architecture enhanced with asynchronous data handling.

Architecture Flow:

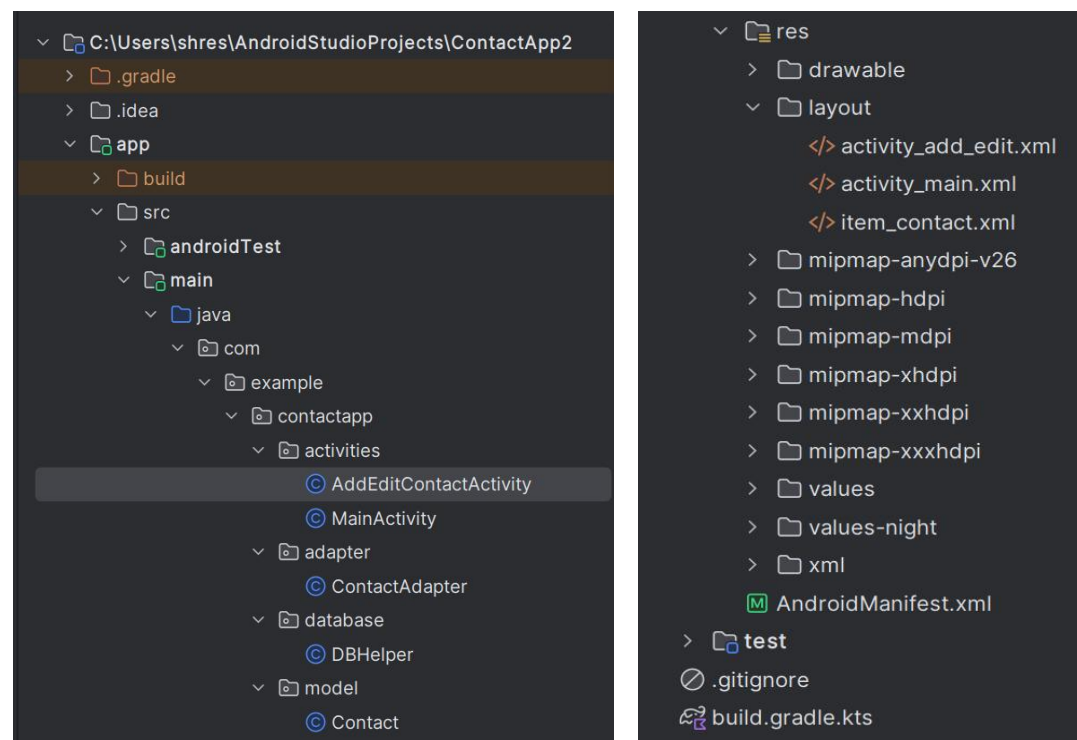
Activity → AppExecutor → SQLite Database

Activity → RecyclerView & Adapter (UI Thread)

Explanation:

- Activities handle user interactions.
- AppExecutor manages background thread execution.
- SQLite database stores contact data locally.
- RecyclerView displays contact data efficiently.
- Background thread operations prevent UI freezing.
- Main thread updates ensure safe UI rendering.
- This architecture strengthens Android fundamentals while applying concurrency best practices.

6. Project Folder Structure



7. Git Workflow and Version Control Strategy

Version 1 of the Smart Contact Management Application was developed using Git for structured version control. All development tasks were committed incrementally to maintain proper tracking of features, improvements, and documentation updates.

7.1 Commit Timeline for V1

The following commits were made on **February 24, 2026**:

Initial Setup Phase

1. Initial commit

Created base Android project structure.

Added initial files and folder hierarchy.

2. Update README with app description

Added basic project overview and description in README file.

UI Enhancements & Layout Improvements

3. Improved UI structure

Refactored layout files for better alignment and structure.

4. Added margin

Improved spacing and layout consistency.

5. Added underline to separate contacts

Added visual divider below contact items for better readability.

6. Improved UI structure (Refinement)

Further optimized layout components and UI consistency.

Feature Enhancements

7. Added alphabetical sorting

Implemented SQL-based sorting using:

ORDER BY name COLLATE NOCASE ASC

Ensured contacts display in A–Z order regardless of letter case.

Repository Synchronization

8. Merge remote changes

Integrated updates from remote repository to maintain version consistency.

Documentation Enhancement

9. Enhance README with features and project details

Expanded README with complete feature list and project explanation.

Prepared documentation for Version 1 submission.

7.2 Version Control Strategy

The following Git practices were followed:

- Incremental commits for each logical change.
- Clear and descriptive commit messages.
- Separation of UI improvements and feature implementations.
- Proper synchronization with remote repository.
- Documentation updates committed separately.
- Verified commits for repository authenticity.
- This structured workflow ensured traceability, organized development, and proper version tracking throughout Version 1.

8. Implementation Details

8.1 Asynchronous Execution (AppExecutor)

A Singleton class named AppExecutor was implemented to manage background and main thread execution. It uses Executors to separate disk I/O tasks from UI updates.

diskIO() → Executes database operations in background.

mainThread() → Updates UI safely on main thread.

This ensures the application remains responsive during database operations.

8.2 Main Activity

The MainActivity loads contacts asynchronously.

Process:

1. Background thread fetches contact list from database.
2. Main thread updates RecyclerView adapter.
3. UI refreshes smoothly without freezing.

8.3 Add/Edit Contact Activity

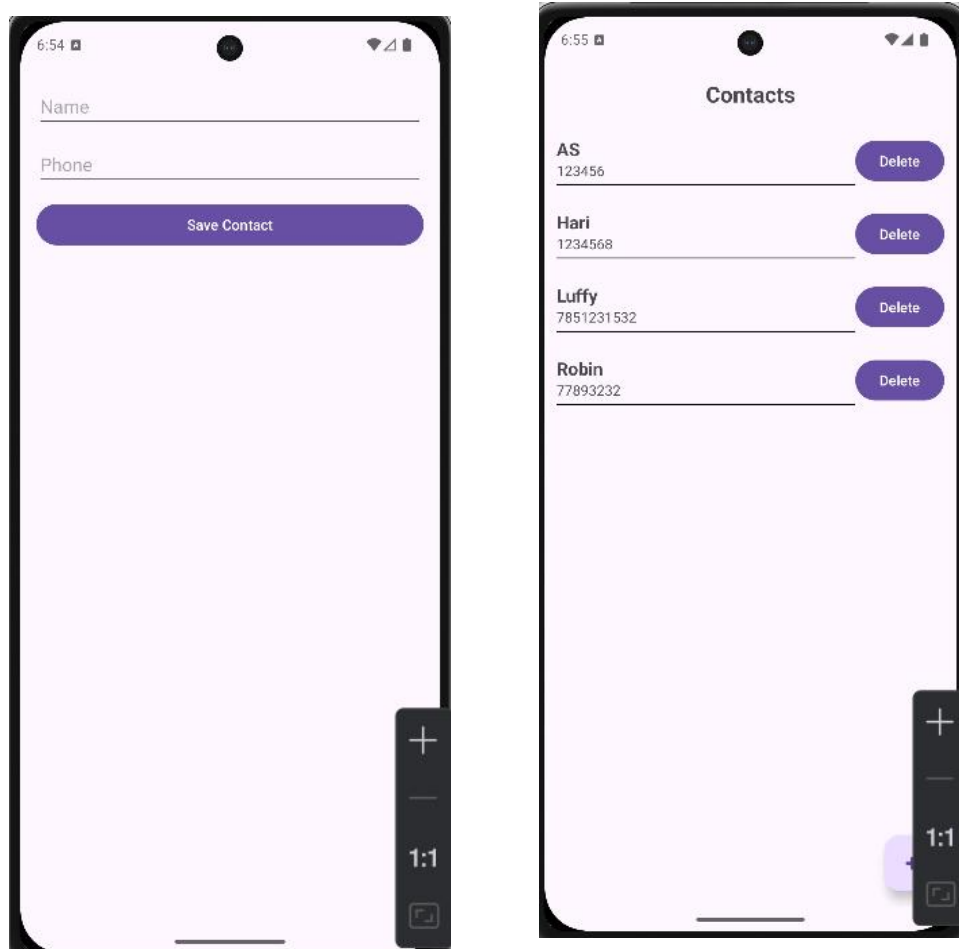
Contact saving and updating operations are also executed in background threads.

Process:

1. Validate user input.
2. Execute insert/update operation using diskIO().
3. Switch back to mainThread().
4. Display success message and finish activity.

9. Screenshots

This is final version 1 of smart contact app.



10. Conclusion

Version 1 of the Smart Contact Management Android Application has been successfully developed into a production-ready state. The application fulfills all defined objectives including:

- Complete CRUD functionality
- Expanded data model
- Material Design-based UI
- Asynchronous database execution

The implementation of background thread handling ensures smooth performance and enhanced user experience. The project demonstrates proper Android development practices and structured version control management.

11. Future Work

Version 2:

- Implement an "Empty State" UI for no contacts.
- Migrate architecture to MVVM pattern.
- Improve UI animations and transitions.

Version 3:

- Caller identification feature.
- Spam detection system.
- Cloud synchronization and online backup.
- Integration with system contacts.